Solitaire Game

In Java

Study Course

B198c9 End-to-End Software Engineering Project

By

Chehab Hany Mohamed Elsayed Elsayed Elmenoufi

Student Number: GH1034223

Under the Guidance of

Prof. Mazhar Hameed

GitHub URL: https://github.com/Chippo90/End-to-End-Software-Engineering-Project

Video Recording URL: https://youtu.be/uCu2LgxTP3E

Gisma University of Applied Sciences

Berlin, Germany

June 2026

Table of Contents

Abstract 3

1. Introduction 4

2. System Architecture 5

2.1 Presentation Layer "UI" 5

2.2 Logic Layer "Game Engine" 5

2.3 Data Layer "Foundation" 6

2.4 Class Diagram 6

3. Implementation 7

3.1 Presentation Layer 7

3.2 Logic Layer 8

3.3 Data Layer 10

3.4 Technologies Used 13

4. Results 14

4.1 Overview 14

4.1.1 Manual Gameplay 14

4.1.2 AI-Assisted Gameplay 14

4.2 Screenshots 14

4.2.1 Main Game 14

4.2.2 During Game – Manual Gameplay 15

4.2.3 During Game – Assisted Gameplay 15

4.2.4 Winning - Assisted Gameplay 16

4.2.5 No More Moves – Manual Gameplay 16

5. Challenges and Solutions 17

5.1 Challenge #1: State Corruption on "New Game" 17

5.2 Challenge #2: State Desynchronization During Drag & Drop 17

5.3 Challenge #3: Differentiating Single vs Stack Drags 17

6. Conclusion and Future Work 18

7. References 19

Abstract

This report shows the implementation of a Solitaire game developed in Java. The project goals were to create a playable Solitaire application and integrate an AI assistant to provide user with recommendations. The system was built using a three-layer design *to ensure a clean separation between the user interface, the core of game, and the data structures. The AI was implemented using a heuristic based scoring algorithm that evaluates all possible moves and suggests the best one. The application is functional and shows the benefits of layered architecture.*

1. Introduction

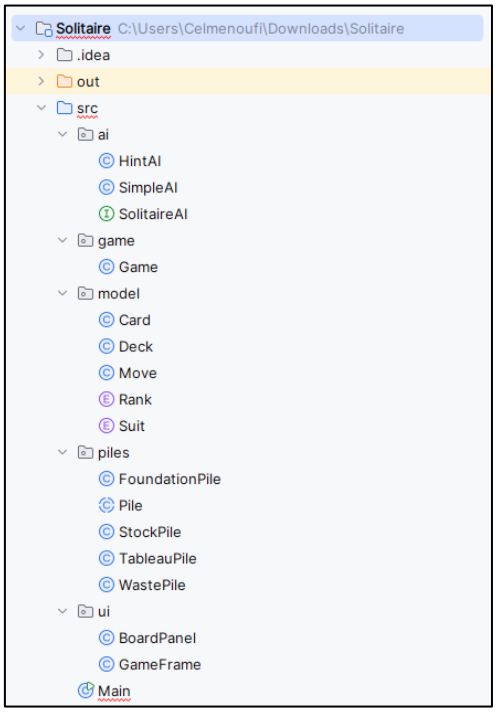
Solitaire is a classic single player card game played by many people. While it is simple to learn, the game involves a level of strategy. The project is to develop a digital version of Solitaire with modern software architecture and an intelligent feature; an AI hint system.

The objective was not just to create a game but to build it with software principles that ensure maintainability, testability, and scalability. A layered architecture was chosen as the architectural foundation. This approach prevents the common problem of hard to manage code by separating the application into distinct layers, each with a specific responsibility.

This report shows the system architecture by explaining the role of each layer. Then covers the implementation of the game's logic, user interface, and heuristic AI. After that, an analysis of the game's performance with and without AI assistance will be presented. The report also discusses the challenges faced during development. Finally, it concludes with a summary of the project's success and suggests possibilities for future work.

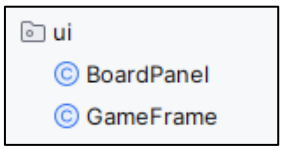
2. System Architecture

The project is structured as a three-layer application. This design is crucial for managing complexity, as it ensures that each part of the system can be developed and tested independently.(“Solitaire Game Development Guide - Ultimate Process - Riseup Labs,” 2022)



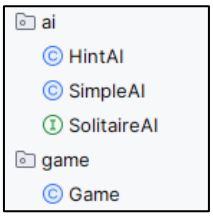
2.1 Presentation Layer “UI”

- **Responsibility**: Presenting the game state to the user and capturing the input. It has no control over the game rules. (JustAcademy, no date)
- **Components**: **“GameFrame”** is the main window and **“BoardPanel”** is the place where the game is drawn. The BoardPanel implements “Mouse Listener” to capture mouse clicks and drags and transform them into commands for the logic layer.(*MouseListener and MouseMotionListener in Java*, 15:54:48+00:00)



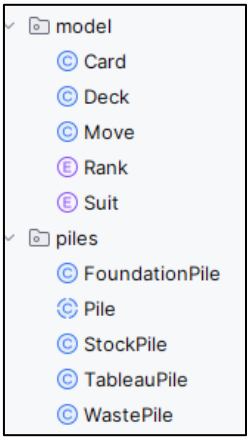
2.2 Logic Layer “Game Engine”

- **Responsibility**: Acting as the brain of the application. It contains the core game logic, processes command from the Presentation Layer and manipulates the Data Layer according to the game's rules. (*Business-Logic Layer*, 13:09:35+00:00)
- **Components**:
  - “**Game**” class controls the entire game, holding references to all the data objects and having the master “executeMove” method.
  - “**HintAI**” makes decisions based on the overall game state.
  - **“SolitaireAI”** defines what any AI for the game must be able to do. In this case, it says that any AI must have a “findBestMove” method.
  - **“SimpleAI”** logic might be to just find the first legal move, without scoring or comparing options. It's dumb but functional AI.

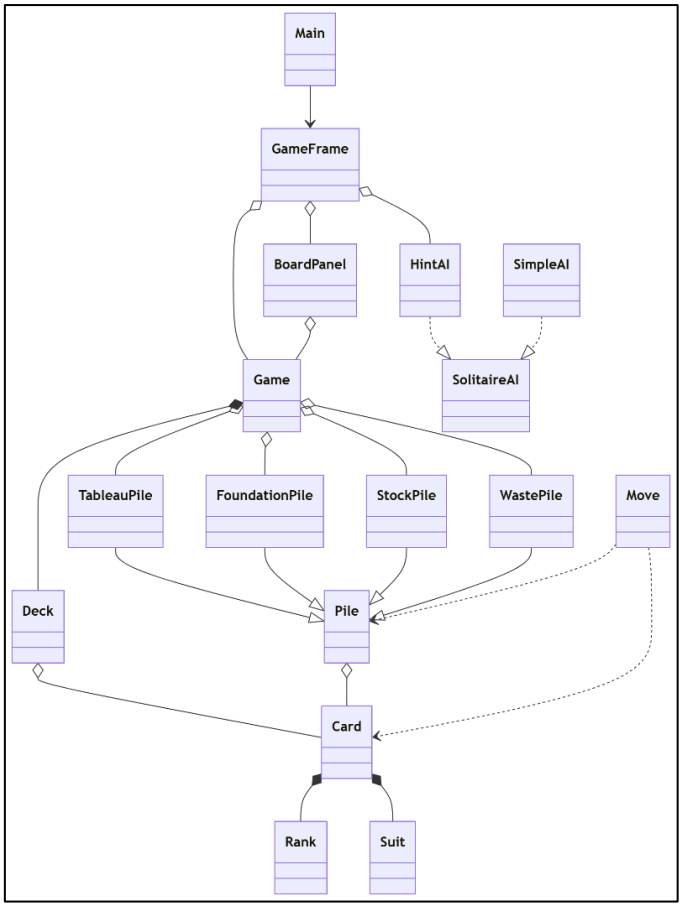


2.3 Data Layer "Foundation"

- **Responsibility**: Representing the data structures and rules. It defines what a card is, what a deck is, and the rules for placing a card in piles. (baeldung, 2018)
- **Components**: "**Card**", "**Deck**", "**Suit**", "**Rank**", "**Move**", and all "**Piles**" subclasses.



2.4 Class Diagram



3. Implementation

The system was developed in Java using the Swing library for the Presentation Layer.

3.1 Presentation Layer

The "**GameFrame**" sets the main window and control buttons. The "**BoardPanel**" is responsible for user interaction.

- "paintComponent" method reads the state from the "Game" object and draws the cards in piles. (*How can we implement the paintComponent() method of a JPanel in Java?*, no date)

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    for (Pile p : game.getAllPiles()) {
        p.draw(g);
    }
    if (draggedCards != null && !draggedCards.isEmpty() && getMousePosition() != null) {
        int currentX = getMousePosition().x - dragOffsetX;
        int currentY = getMousePosition().y - dragOffsetY;
        for (Card card : draggedCards) {
            if (card.isFaceUp()) {
                g.setColor(Color.WHITE);
                g.fillRect(currentX, currentY, Pile.CARD_WIDTH, Pile.CARD_HEIGHT);
                g.setColor(card.getColor());
                g.drawString(card.getShortName(),  x: currentX + 5,  y: currentY + 20);
                g.setColor(Color.BLACK);
                g.drawRect(currentX, currentY, Pile.CARD_WIDTH, Pile.CARD_HEIGHT);
            }
            currentY += 20;
        }
    }
}
```

- To solve bugs, the drag and drop was implemented using a "visual copy" approach. When a user "mousePressed" on a card, the BoardPanel creates a temporary copy of the card to be dragged.(*What's the difference between mouseClicked and mousePressed. (Swing / AWT / SWT forum at Coderanch)*, no date)

```
@Override
public void mousePressed(MouseEvent e) {
    int x = e.getX();
    int y = e.getY();
    if (game.getStockPile().includes(x, y)) {
        if (game.getStockPile().isEmpty()) {
            game.recycleWaste();
        } else {
            game.drawFromStock();
        }
        repaint();
        return;
    }

    for (Pile p : game.getAllPiles()) {
        if (p.includes(x, y) && !p.isEmpty()) {
            sourcePileForDrag = p;
            if (p instanceof TableauPile) {
                Card clickedCard = ((TableauPile) p).getCardAt(y);
                if (clickedCard != null && clickedCard.isFaceUp()) {
                    draggedCards = new Stack<>();
                    int index = p.getCards().indexOf(clickedCard);
                    for (int i = index; i < p.getCards().size(); i++) {
                        draggedCards.push(p.getCards().get(i));
                    }
                }
            } else {
                draggedCards = new Stack<>();
                draggedCards.push(p.peek());
            }
            if (draggedCards != null && !draggedCards.isEmpty()) {
                dragOffsetX = x - p.getX();
                dragOffsetY = y - (p.getY() + (p.size() - draggedCards.size()) * 20);
                break;
            }
        }
    }
}
```

- When the “mouseReleased” event happens, the BoardPanel checks if the move is valid by calling the canAccept method on the destination pile. If the move is valid, it sends a command to the Game “executeMove” method to change the state. (*MouseListener and MouseMotionListener in Java*, 15:54:48+00:00)

```java
@Override
public void mouseReleased(MouseEvent e) {
    if (draggedCards == null || draggedCards.isEmpty()) {
        return;
    }
    Pile destinationPile = null;
    for (Pile p : game.getAllPiles()) {
        if (p != sourcePileForDrag && p.includes(e.getX(), e.getY())) {
            destinationPile = p;
            break;
        }
    }
    Card cardToMove = draggedCards.firstElement();
    if (destinationPile != null && destinationPile.canAccept(cardToMove)) {
        game.executeMove(new Move(sourcePileForDrag, destinationPile, cardToMove));
    }
    draggedCards = null;
    sourcePileForDrag = null;
    repaint();
    if (game.checkForWin()) {
        JOptionPane.showMessageDialog( parentComponent: this,  message: "Congratulations, you won!");
    }
}
```

3.2 Logic Layer

- “**Game**” class acts as the central hub. It holds instances of all the piles and the deck.

```java
public class Game {  16 usages
    private final List<Pile> allPiles;  6 usages
    private final StockPile stockPile;  8 usages
    private final WastePile wastePile;  6 usages
    private final List<FoundationPile> foundationPiles;  4 usages
    private final List<TableauPile> tableauPiles;  5 usages
```

- “executeMove” method is the single entry for changing the game state. This method is handling moves of single cards from the “WastePpile” and multi card stacks from “TableauPile” through a logic that calls the split method on the “TableauPile”.

```java
public void executeMove(Move move) {  1 usage
    Stack<Card> cardsToMove;
    if (move.sourcePile instanceof TableauPile) {
        cardsToMove = ((TableauPile) move.sourcePile).split(move.card);
    } else {
        cardsToMove = new Stack<>();
        Card c = move.sourcePile.pop();
        if (c != null) cardsToMove.push(c);
    }

    if (cardsToMove != null && !cardsToMove.isEmpty()) {
        for (Card card : cardsToMove) {
            move.destinationPile.push(card);
        }
    }

    if (move.sourcePile instanceof TableauPile) {
        Pile source = move.sourcePile;
        if (!source.isEmpty() && !source.peek().isFaceUp()) {
            source.peek().setFaceUp(true);
        }
    }
}
```

- "**HintAI**" was also implemented in this layer. It operates by generating a list of all possible legal moves from the current game state. It then evaluates each move against a heuristic scoring function, which assigns points based on the move.

```
private int calculateMoveScore(Move move, Game game) {  1 usage
    int score = 0;

    //Highest priority: Moving any card to the foundation is always good.
    if (move.destinationPile instanceof FoundationPile) {
        score += 100;
    }

    //Uncovering a face-down card in the tableau is very good.
    if (move.sourcePile instanceof TableauPile) {
        TableauPile source = (TableauPile) move.sourcePile;
        // Check if this move would uncover a face-down card
        if (!source.isEmpty() && source.peek() != move.card && !source.peek().isFaceUp()) {
            score += 50;
        }
    }

    //Moving a King to an empty tableau spot is good.
    if (move.destinationPile instanceof TableauPile) {
        if (((TableauPile) move.destinationPile).isEmpty() && move.card.getRank().getValue() == 13) {
            score += 25;
        }
    }

    //Moving from the waste pile is a standard progress move.
    if (move.destinationPile instanceof TableauPile) {
        score += 10;
    }

    //Drawing from the stock is a last resort, low score.
    if (move.sourcePile == game.getStockPile()) {
        score += 1;
    }

    return score;
}
```

- Finally, it selects and returns the move with the highest score. It is a tactical AI, focusing on the best immediate action rather than long-term strategy.

```
private List<Move> findAllPossibleMoves(Game game) {  1 usage
    List<Move> moves = new ArrayList<>();
    Card wasteCard = game.getWastePile().peek();

    //1. Check moves from Waste to Foundation/Tableau
    if (wasteCard != null) {
        for (FoundationPile p : game.getFoundationPiles()) {
            if (p.canAccept(wasteCard)) moves.add(new Move(game.getWastePile(), p, wasteCard));
        }
        for (TableauPile p : game.getTableauPiles()) {
            if (p.canAccept(wasteCard)) moves.add(new Move(game.getWastePile(), p, wasteCard));
        }
    }

    //2. Check moves from Tableau to Foundation/other Tableaus
    for (TableauPile source : game.getTableauPiles()) {
        if (!source.isEmpty()) {
            Card topCard = source.peek();
            //To Foundation
            for (FoundationPile p : game.getFoundationPiles()) {
                if (p.canAccept(topCard)) moves.add(new Move(source, p, topCard));
            }
            //To other Tableaus (moving single card)
            for (TableauPile dest : game.getTableauPiles()) {
                if (source != dest && dest.canAccept(topCard)) moves.add(new Move(source, dest, topCard));
            }
            //Check for moving sub-stacks
            for(Card card : source.getCards()) {
                if (card.isFaceUp() && card != topCard) {
                    for (TableauPile dest : game.getTableauPiles()) {
                        if (source != dest && dest.canAccept(card)) moves.add(new Move(source, dest, card));
                    }
                }
            }
        }
    }

    //3. Check for drawing from stock
    if (!game.getStockPile().isEmpty()) {
        moves.add(new Move(game.getStockPile(), game.getWastePile(), game.getStockPile().peek()));
    } else if (!game.getWastePile().isEmpty()) {
        moves.add(new Move(game.getWastePile(), game.getStockPile(), game.getWastePile().peek()));
    }

    return moves;
}
```

3.3 Data Layer

The foundation of the game lies in its data objects.

- "**Suit**" and "**Rank**" were implemented as enums for safety and clarity. (*Java Enums*, no date)

```
package model;

/**
 *An enum representing the four suits in a standard deck of cards.
 *Enums are used here because the set of suits is fixed and known.
 */
public enum Suit {  7 usages
    HEARTS, DIAMONDS, CLUBS, SPADES  2 usages
}
```

```java
package model;

/**
 *An enum representing the 13 ranks in a standard deck.
 *Each rank is given a numerical value to make rule-checking easier.
 */
public enum Rank {  9 usages
    ACE( value: 1), TWO( value: 2), THREE( value: 3), FOUR( value: 4), FIVE( value: 5), SIX( value: 6), SEVEN( value: 7),
    EIGHT( value: 8), NINE( value: 9), TEN( value: 10), JACK( value: 11), QUEEN( value: 12), KING( value: 13);  no usages

    private final int value; // The integer value of the rank.  2 usages

    /**
     *Constructor for the enum.
     */
    Rank(int value) { this.value = value; }

    public int getValue() { return value; }
}
```

- "**Card**" class combines these enums. (*enum in Java*, 10:17:33+00:00)

```java
public class Card {
    private final Suit suit;  6 usages
    private final Rank rank;  5 usages
    private boolean isFaceUp;  3 usages

    public Card(Suit suit, Rank rank) {
        this.suit = suit;
        this.rank = rank;
        this.isFaceUp = false; // Cards start face-down by default.
    }
```

- "**Deck**" class was implemented to guarantee that the constructor generates a complete set of 52 cards, which is then shuffled.

```java
public Deck() {  1 usage
    //Initialize a new, empty stack to hold the cards.
    this.cards = new Stack<>();

    //This programmatically builds a complete set of 52 cards.
    for (Suit suit : Suit.values()) {
        for (Rank rank : Rank.values()) {
            // Add one of each card to the deck.
            cards.push(new Card(suit, rank));
        }
    }

    // Shuffle the newly created, perfect deck.
    shuffle();
}

/**
 *Randomizes the order of the cards in the deck.
 */
public void shuffle() {  1 usage
    Collections.shuffle(cards);
}
```

- The abstract "**Pile**" class defines common behaviors, while each subclass provides a concrete implementation of the "canAccept" method, which enforces the rules of Solitaire.

```
public abstract class Pile {  23 usages  4 inheritors
    // The physical location and size of the pile on the screen.
    protected int x;  25 usages
    protected int y;  22 usages
    public static final int CARD_WIDTH = 71;  20 usages
    public static final int CARD_HEIGHT = 96;  21 usages

    //Inside the Pile class, add these two methods:
    public int getX() {  1 usage
        return x;
    }
    public int getY() {  1 usage
        return y;
    }
    //The stack holding the cards. 'protected' so subclasses can access it.
    protected final Stack<Card> cards = new Stack<>();  19 usages

    public Pile(int x, int y) {  4 usages
        this.x = x;
        this.y = y;
    }
```

- "**FoundationPile**" class uses "canAccept" method checks if a card is an or if it is of the same suit and one rank higher than the current top card.

```
public boolean canAccept(Card card) {
    if (isEmpty()) {
        return card.getRank() == Rank.ACE;
    }
    Card topCard = peek();
    return topCard.getSuit() == card.getSuit() &&
            topCard.getRank().getValue() + 1 == card.getRank().getValue();
}
```

- "**TableauPile**" class implements the game rules. "canAccept" method verifies if a card is a King or if it is of the opposite color and one rank lower than the current top card.

```
public boolean canAccept(Card card) {
    if (isEmpty()) {
        return card.getRank() == Rank.KING;
    }
    Card topCard = peek();
    if(topCard == null || card == null) return false;
    return topCard.getColor() != card.getColor() &&
            topCard.getRank().getValue() - 1 == card.getRank().getValue();
}
```

- "**StockPile**" and "**WastePile**" classes represent a special case. "canAccept" methods always return false, as cards are never moved to them through a standard player drag and drop.

```
public boolean canAccept(Card card) {
    return false;
}
```

Their state is managed by special methods in the Logic Layer "**drawFromStock**" and "**recycleWaste**". (Orozco-Fletcher, 2021; *Why it is important to use the Recycle() method on every Java object*, no date)

```
public void draw(Graphics g) {
    if (isEmpty()) {
        // --- FIX: Use a contrasting color for the empty placeholder ---
        g.setColor(Color.DARK_GRAY);
        g.drawRect(x, y, CARD_WIDTH, CARD_HEIGHT);
        g.drawString( str: "R",  x: x + CARD_WIDTH / 2 - 4,  y: y + CARD_HEIGHT / 2 + 5);
    } else {
        // Draw the back of a card
        g.setColor(Color.BLUE);
        g.fillRect(x, y, CARD_WIDTH, CARD_HEIGHT);
        g.setColor(Color.WHITE);
        g.drawRect(x, y, CARD_WIDTH, CARD_HEIGHT);
    }
}
```

```
public void recycle(Stack<Card> wasteCards) {
    while (!wasteCards.isEmpty()) {
        Card card = wasteCards.pop();
        card.setFaceUp(false);
        this.push(card);
    }
}
```

3.4 Technologies Used

- **Language**: Java
- **Framework**: Swing (*Introduction to Java Swing*, 14:17:44+00:00)
    - JFrame: build the application window
    - JButton: control buttons
    - JLabel: status labels
- **Diagram**: Mermaid.js

4. Results

4.1 Overview

To evaluate the effectiveness of the “HintAI”, gameplay was compared across two modes: manual play and strictly following the AI's suggestions.

4.1.1 Manual Gameplay

In this mode, success is entirely dependent on the player and the luck of the deal. It is easy for a player to miss an optimal move, such as clearing a tableau column that would reveal a crucial card. This can often lead to getting stuck in a solvable game.

Out of 5 played games, I was able to win only 1 game. The remaining games I reached the “No More Moves” situation.

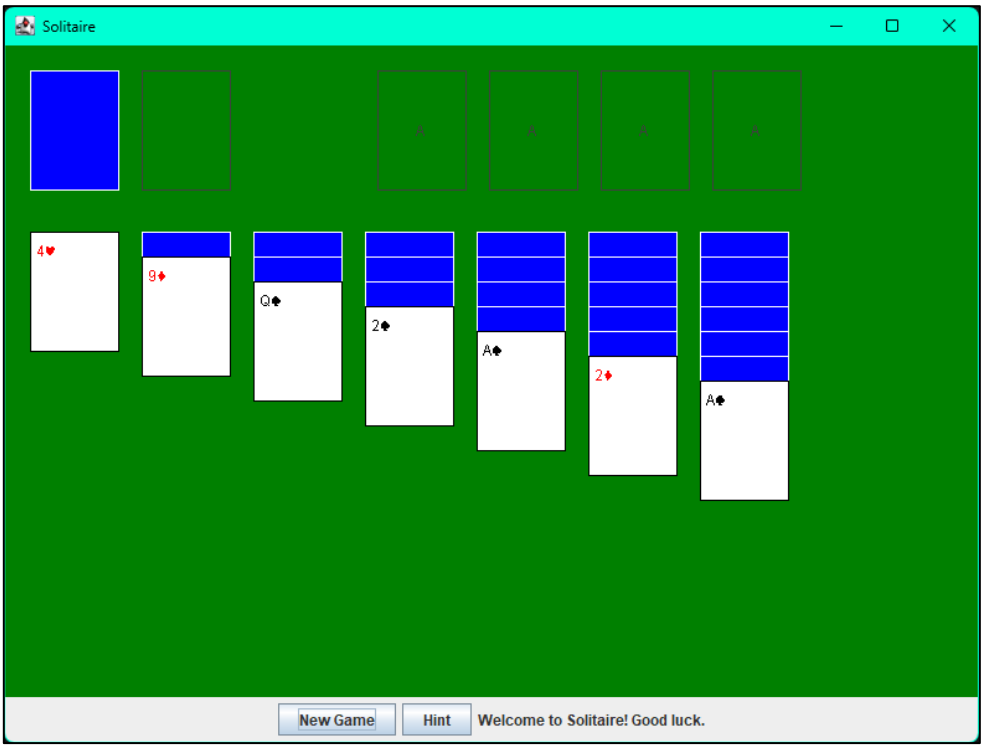
4.1.2 AI-Assisted Gameplay

By following the AI's hints, gameplay becomes highly efficient. The AI, with its perfect view of the board, never misses an opportunity to move a card to the foundation or uncover a card. The result is tactical where games are often completed faster and the win rate for solvable deals is significantly higher.

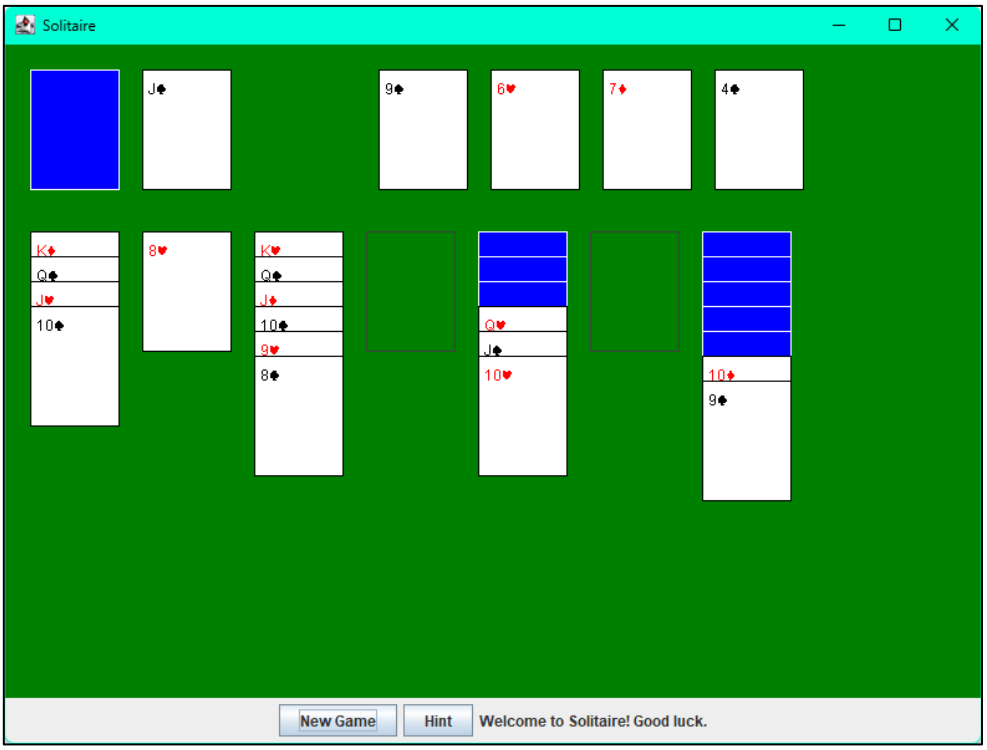
Out of 5 played games, I was able to win 4 games. I was completely pressing the “Hint” button on every move.

4.2 Screenshots

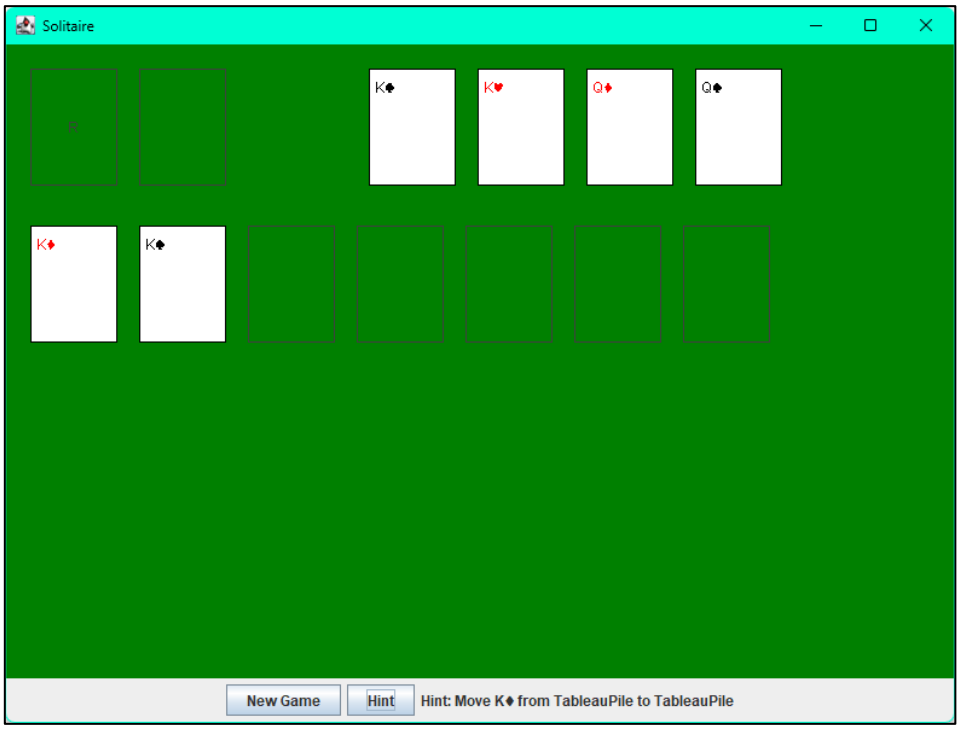
4.2.1 Main Game



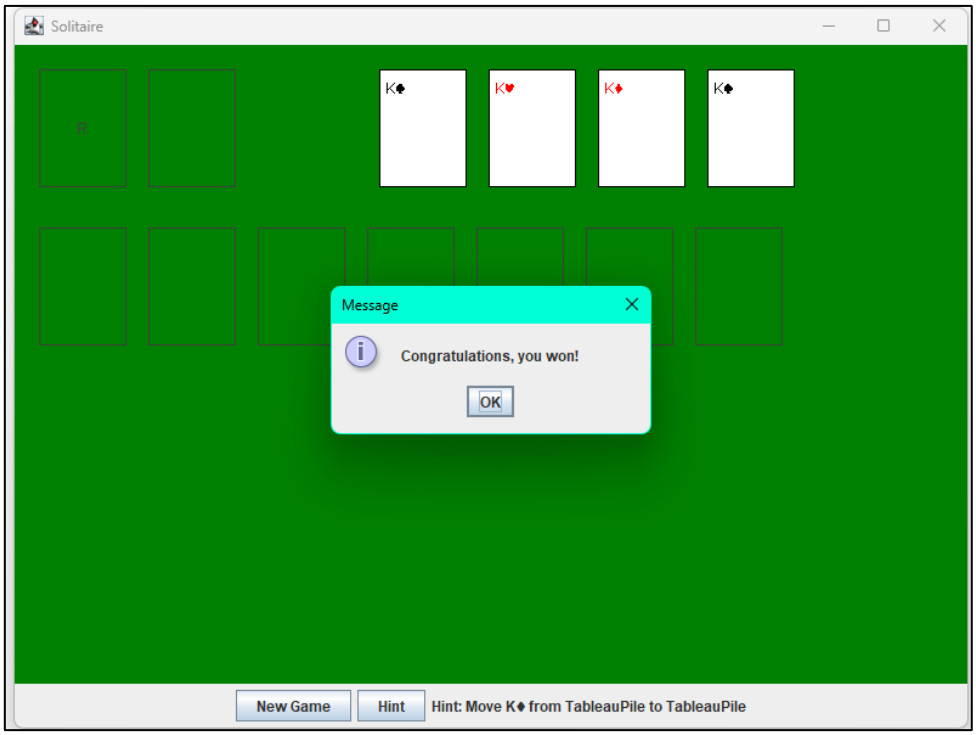
4.2.2 During Game – Manual Gameplay



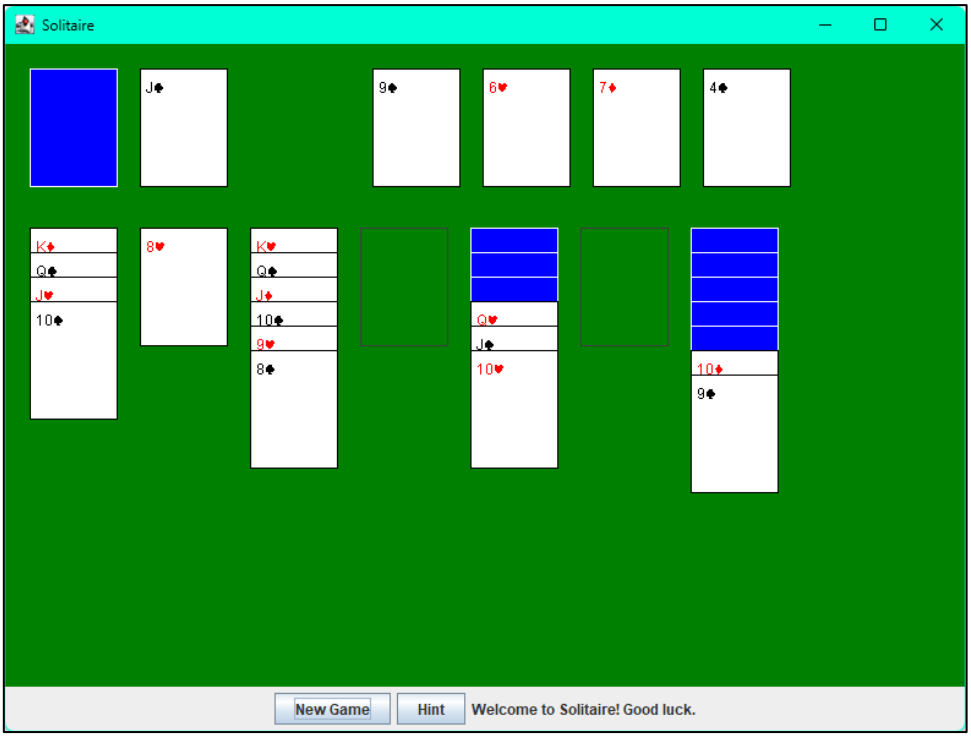
4.2.3 During Game – Assisted Gameplay



4.2.4 Winning - Assisted Gameplay



4.2.5 No More Moves – Manual Gameplay



5. Challenges and Solutions

5.1 Challenge #1: State Corruption on "New Game"

- **Description**: A critical bug caused the game to fail when starting a new game. The board would render with empty piles, duplicate cards, or a mixture of old and new game states.
- **Solution**: The "New Game" button, part of the Presentation Layer, was implemented to discard the old logic “Game” object entirely and create a brand new, clean instance. This guarantees a perfect, uncorrupted state for every new game.

5.2 Challenge #2: State Desynchronization During Drag & Drop

- **Description**: The most frustrating bug involved cards disappearing or moves failing after being dragged. This occurred because the “BoardPanel” was incorrectly modifying the data layer directly by removing cards from piles during the drag operation.
- **Solution**: The drag and drop logic in “BoardPanel” was rewritten to use the "visual copy" approach. The Presentation Layer now never touches the real cards. It only creates a temporary visual clone for dragging. The actual state change is handled exclusively by the game logic layer's “executeMove” method after a move is validated.

5.3 Challenge #3: Differentiating Single vs Stack Drags

- **Description**: To correctly handle drag and drop actions on the tableau piles. Unlike simple single-card piles like Waste or Foundation, a tableau pile can have multiple face-up cards stacked on top of each other. The system needed to differentiate between a user wanting to move only the top card versus wanting to move an entire valid sub-stack.
- **Solution**: The problem was solved by encapsulating the complex hit detection logic within the “TableauPile” class itself. Two methods were created:(*Solved Use java Language to program solitaire: Program uses | Chegg.com*, no date)

    - **getCardAt:** This method was added to the “TableauPile” class. It takes the y-coordinate of the mouse click and iterates through the cards in its own stack, calculating positions to determine which card the user clicked on. (Patel, 2015)

```
public Card getCardAt(int pY) {  1 usage
    for (int i = cards.size() - 1; i >= 0; i--) {
        Card card = cards.get(i);
        int cardTopY = y + i * Y_OFFSET;
        int cardBottomY = (i == cards.size() - 1) ? cardTopY + CARD_HEIGHT : cardTopY + Y_OFFSET;
        if (pY >= cardTopY && pY < cardBottomY) {
            return card;
        }
    }
    return null;
}
```

- **split:** Once the clicked card is identified, this method is called. It programmatically splits the pile at that card, returning a new stack containing the clicked card and all the cards below it.(Marcus, 2013)

```
public Stack<Card> split(Card card) {  1 usage
    int splitIndex = cards.indexOf(card);
    if (splitIndex == -1) {
        return new Stack<>();
    }

    Stack<Card> movedCards = new Stack<>();
    movedCards.addAll(cards.subList(splitIndex, cards.size()));

    cards.subList(splitIndex, cards.size()).clear();

    return movedCards;
}
```

6. Conclusion and Future Work

This project successfully achieved its goal of creating a functional GUI Solitaire game with an intelligent AI hint system. The adoption of layered architecture, while presenting initial challenges, proved the project's success, allowing for the resolution of complex bugs. The application is a stable, playable game where heuristic AI provides genuinely useful tactical advice.

Future Work

The current application provides a solid foundation for many exciting enhancements:

- **Advanced AI:** Implementing a search-based algorithm that can look multiple moves ahead to provide long-term strategic advice.
- **Undo/Redo Functionality:** Adding a feature to undo moves would greatly improve player experience.
- **Timer & Score:** Adding timer along with scoring board, since both are measuring the best scores.
- **UI/UX Enhancements:** Improving the visual assets with high-resolution card graphics and adding smooth animations for card movements.

---

7. References

baeldung (2018) *The DAO Pattern in Java | Baeldung*. Available at: https://www.baeldung.com/java-dao-pattern (Accessed: June 20, 2026).

Business-Logic Layer (13:09:35+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/dbms/business-logic-layer/ (Accessed: June 20, 2026).

enum in Java (10:17:33+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/java/enum-in-java/ (Accessed: June 20, 2026).

How can we implement the paintComponent() method of a JPanel in Java? (no date). Available at: https://www.tutorialspoint.com/article/how-can-we-implement-the-paintcomponent-method-of-a-jpanel-in-java (Accessed: June 20, 2026).

Introduction to Java Swing (14:17:44+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/java/introduction-to-java-swing/ (Accessed: June 20, 2026).

Java Enums (no date). Available at: https://www.w3schools.com/java/java_enums.asp (Accessed: June 20, 2026).

JustAcademy (no date) *Java presentation layer by Roshan Chaturvedi | JustAcademy*, *Just Academy*. Available at: https://www.justacademy.co/blog-detail/java-presentation-layer (Accessed: June 20, 2026).

Marcus (2013) "How to Split a deck of cards in half?," *Stack Overflow*. Available at: https://stackoverflow.com/q/20480243 (Accessed: June 20, 2026).

MouseListener and MouseMotionListener in Java (15:54:48+00:00) *GeeksforGeeks*. Available at: https://www.geeksforgeeks.org/java/mouselistener-mousemotionlistener-java/ (Accessed: June 20, 2026).

Orozco-Fletcher, M. (2021) "Java Lesson 21: Drawing and Coloring Shapes on the JFrame," *Medium*, 16 December. Available at: https://medium.com/@michael71314/java-lesson-21-drawing-and-coloring-shapes-on-the-jframe-d740970e1d68 (Accessed: June 20, 2026).

Patel, H. (2015) "Java - Creating a Card Deck Class," *Stack Overflow*. Available at: https://stackoverflow.com/q/27889641 (Accessed: June 20, 2026).

"Solitaire Game Development Guide - Ultimate Process - Riseup Labs" (2022), 16 June. Available at: https://riseuplabs.com/solitaire-game-development-guide/ (Accessed: June 20, 2026).

Solved Use java Language to program solitaire: Program uses | Chegg.com (no date). Available at: https://www.chegg.com/homework-help/questions-and-answers/use-java-language-program-solitaire-program-uses-stacks-queues-deck-waste-tableaus-foundat-q44616695#question-transcript (Accessed: June 20, 2026).

What's the difference between mouseClicked and mousePressed. (Swing / AWT / SWT forum at Coderanch) (no date). Available at: https://coderanch.com/t/333152/java/difference-mouseClicked-mousePressed (Accessed: June 20, 2026).

Why it is important to use the Recycle() method on every Java object (no date). Available at: https://support.hcl-software.com/csm?id=kb_article&sysparm_article=KB0037358 (Accessed: June 20, 2026).